Solution Requirements

Date	2 July 2026
Team ID	xxxxxx
Project Name	Smart Lender
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should *do*).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should *be*).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority (High/Medium/Low)
1	Authentication	The web app does not require user login for the prediction form since it's a public-facing tool. However, if an admin panel is added later, it should use username/password login to restrict access to model management and dataset updates.	Low
2	Authorization levels	Two roles: general user (can access the home page and submit the loan prediction form, view results) and admin (can update the dataset, retrain the model, and view logs). General users have no data modification access	Medium

3	External interfaces	*The system connects to Google Sheets as the source for the loan eligibility dataset. The trained ML model (Scikit-learn/XGBoost) is stored as a .pkl file and loaded by the Flask app at runtime. No payment gateway or external API is required.*	High
4	Transactions processing	The core transaction is the prediction request: the user submits form data (gender, education, income, loan amount, credit history, etc.), the Flask backend processes it through the ML model, and returns an eligibility result (Approved/Not Approved) on the same session. No financial transactions are involved.	High
5	Reporting	The system generates a prediction result page after each form submission. Additionally, EDA visualizations (count plots, distribution plots, bar charts) are generated during the data analysis phase and can be displayed on a dashboard page showing dataset insights.	Medium
6	Business rules, etc.	A loan applicant is eligible only if all required fields are filled (no blanks). Missing numerical values are filled using mean; missing categorical values use mode before prediction. The best-performing model (highest accuracy among Decision Tree, Random Forest, KNN, XGBoost) is the one deployed. The result must be binary: Eligible or Not Eligible.	High

7	Compliance to laws or regulations	laws or regulationsApplicant data entered on the form is not stored or shared — it is used only for the current prediction session. No personally identifiable information (PII) is retained after the session ends, keeping the system broadly aligned with data minimisation principles under GDPR.	Medium
8	*Other*	The system must work on standard desktop browsers (Chrome, Firefox, Edge) without any plugin installation. The Flask app must run locally on Python 3.x. The model file must be pre-trained and saved before the web app is launched.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	SpeedThe Flask web app should respond to a prediction request quickly after the user submits the form. The ML model inference time should be near-instant since it runs locally on a pre-trained model.	Prediction result displayed within 2 seconds of form submission on a standard local machine
2	Scalability	Since this is a locally hosted educational project, scalability is limited. However, the codebase should be structured so it can be deployed to a cloud platform (e.g., Heroku, Render) in the future without major changes.	Supports at least 10–20 concurrent users if deployed; code structured for easy cloud migration

3	Security & Data Privacy	User-submitted form data is not logged or stored. The trained model file (.pkl) should be stored securely and not exposed via any public route. Input fields should be validated to prevent injection or malformed data from reaching the model.	No PII stored after session. All form inputs validated before processing. Model file not directly downloadable via URL.
4	Reliability & Availability	The Flask app should handle incorrect or missing inputs gracefully without crashing — returning a friendly error message instead. The pre-trained model must be loaded successfully on app startup.	Zero unhandled crashes on valid or invalid input. App startup completes with model loaded in under 5 seconds.
5	Usability & Accessibility	The web interface should be clean, simple, and usable without any technical knowledge. Labels for all form fields must be clear.The result page should clearly show Approved or Not Approved with a brief explanation	A first-time user can complete and submit the form within 2 minutes without guidance. All form fields are clearly labelled.
6	*Other*	Code is split into separate files (preprocessing, model training, Flask routes, HTML templates) with comments in key sections. Runs on any 64-bit Windows/Linux system with Python 3.x and required packages installed via pip.	Code split into logical modules. README file included with setup instructions. Runs successfully on a fresh Python environment after pip install.

.